

# Sahayak

Sahayak is a low-bandwidth welfare scheme discovery assistant built for NSS Open Projects 2026 - Track 1.1: AI-Based Multilingual Chatbot for Welfare Scheme Awareness.

The prototype helps a user fill a short eligibility profile, discover relevant welfare schemes, ask grounded follow-up questions, and generate a document checklist in English or Hindi.

Live demo: [https://parzival1821.github.io/Multilingual\\_Chatbot/](https://parzival1821.github.io/Multilingual_Chatbot/)

## Project Snapshot

- Status: Functional MVP deployed on GitHub Pages.
- Scope: 8 high-impact central welfare schemes.
- Languages: English and Hindi, with Hindi rendered in Devanagari.
- Core AI pattern: Retrieval-grounded answer generation over a curated scheme knowledge base.
- User flow: 4-6 simple eligibility signals produce a personalized scheme shortlist.
- Safety: Unknown questions return a fallback instead of hallucinated scheme details.
- Verification: Local core and static smoke tests pass with npm run check.

## NSS Challenge Fit

| Official expectation              | Current implementation                                                                        |
|-----------------------------------|-----------------------------------------------------------------------------------------------|
| Focused catalogue of 8-10 schemes | 8 schemes included                                                                            |
| Eligibility-questioning flow      | Guided profile form with area, occupation, gender, income, land, housing, LPG, bank, and more |
| Personalized shortlist            | Ranked recommendations with match reasons                                                     |
| Multilingual interface            | English + Hindi Devanagari                                                                    |
| Document checklist                | Copyable and downloadable scheme-specific checklist                                           |
| Grounded answers                  | Retrieval over local scheme data with source links                                            |
| Low-bandwidth thinking            | Static app, text-first UI, no heavy framework, downloadable .txt handoff                      |
| Honest unknown handling           | Safe fallback for unsupported questions                                                       |

## Scheme Catalogue

| Scheme                                 | Primary beneficiary                                              | Current verification     |
|----------------------------------------|------------------------------------------------------------------|--------------------------|
| PM-KISAN Samman Nidhi                  | Landholding farmer families                                      | Verified official source |
| Ayushman Bharat PM-JAY                 | Poor and vulnerable families needing health cover                | Verified official source |
| Pradhan Mantri Ujjwala Yojana          | Adult women from poor households without LPG                     | Verified official source |
| Pradhan Mantri Awas Yojana - Urban 2.0 | Urban EWS/LIG/MIG families without a permanent house             | Verified official source |
| National Social Assistance Programme   | BPL elderly, widows, disabled persons, and vulnerable households | Verified official source |
| PM Street Vendor's AtmaNirbhar Nidhi   | Urban and peri-urban street vendors                              | Official source          |
| Atal Pension Yojana                    | Informal/unorganised workers with bank accounts                  | Official source          |
| Sukanya Samriddhi Account              | Girl child, through parent or guardian                           | Official source          |

## System Architecture

Diagram: Rendered in the Visual Diagrams Appendix PDF.

## User Journey

Diagram: Rendered in the Visual Diagrams Appendix PDF.

## Grounded Answer Flow

Diagram: Rendered in the Visual Diagrams Appendix PDF.

## Demo Personas

Diagram: Rendered in the Visual Diagrams Appendix PDF.

## Implementation Details

- index.html and styles.css implement the responsive single-page dashboard.
- src/data.js stores curated scheme data, Hindi names, summaries, labels, documents, benefits, and source links.
- src/core.js contains eligibility scoring, retrieval, profile inference, checklist formatting, and grounded answer composition.
- src/app.js connects the UI to the recommendation engine, assistant, language switcher, and checklist actions.
- tests/core.test.js validates recommendation ranking, retrieval, fallback behavior, Hindi queries, and checklist logic.
- tests/static-smoke.test.js validates the static shell, deployed asset references, Hindi Devanagari copy, and Pages workflow shape.

## Submission Documents

The submission/ folder contains Google-Docs-ready Markdown files for the official NSS deliverables:

- [Combined PDF Submission Packet](#)
- [Problem Analysis Report](#)
- [Solution Framework](#)
- [Prototype and Implementation Plan](#)
- [Impact Projection](#)
- [Pilot Test Report and Internal Validation](#)
- [Final Presentation Outline](#)
- [Submission Portal Details](#)
- [Visual Diagrams Appendix](#)

## Run Locally

npm run dev

Then open:

<http://localhost:4173>

## Test

npm test

Run all local checks:

npm run check

## Demo Script

1. Open the live app or run it locally.
2. Click the Farmer preset.
3. Show PM-KISAN as a strong match and PM-JAY as an additional support option.
4. Click View documents on PM-KISAN and show the checklist panel.
5. Copy or download the checklist to demonstrate low-bandwidth handoff.
6. Ask: What documents do I need for Ayushman Bharat?

7. Switch to Hindi and ask: मुझे घर के लिए कौन सी योजना मिल सकती है?
8. Show source links, verification labels, and the safe fallback for unsupported questions.

## Verification Checklist

- Farmer persona ranks PM-KISAN first.
- Woman-led household persona shows Ujjwala, PMAY-U, and Sukanya.
- Street vendor persona ranks PM SVANidhi first.
- Hindi mode uses Devanagari labels, prompts, scheme names, and checklist content.
- Recommendation View documents action scrolls to and highlights the checklist panel.
- Checklist copy and download actions produce a scheme-specific document list.
- Unsupported questions produce a safe fallback instead of invented scheme details.
- `npm run check` passes.
- `details.md`, `plan.md`, and `schemes.md` remain local-only and uncommitted.

## Project Positioning

Sahayak is intentionally built as a practical AI product rather than a custom ML research model. Its strength is the combination of structured eligibility logic, retrieval-grounded answers, bilingual UX, source-backed recommendations, and a low-bandwidth document handoff that can later be extended to WhatsApp, SMS, or IVR.

## Submission Documents

This folder contains Google-Docs-ready submission material for Sahayak, prepared for NSS Open Projects 2026 - Track 1.1: AI-Based Multilingual Chatbot for Welfare Scheme Awareness.

These files are written in Markdown so they render cleanly on GitHub and can be copied into Google Docs for final formatting.

### Document Index

| File                                                | Purpose                                                                                       |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <a href="#">01-problem-analysis-report.md</a>       | Problem framing, root causes, stakeholders, personas, and success criteria                    |
| <a href="#">02-solution-framework.md</a>            | Architecture, process flow, tech stack, safety, scalability, and risk mitigation              |
| <a href="#">03-prototype-implementation-plan.md</a> | Current MVP, feature map, deployment, testing, and implementation roadmap                     |
| <a href="#">04-impact-projection.md</a>             | Quantified impact model, assumptions, scenarios, and sustainability pathway                   |
| <a href="#">05-pilot-test-report.md</a>             | Internal validation report with scripted personas, test results, and real-user pilot appendix |
| <a href="#">06-final-presentation-outline.md</a>    | 8-slide pitch deck outline with speaker notes                                                 |
| <a href="#">07-submission-portal-details.md</a>     | Exact portal fields, links, and final submission checklist                                    |
| <a href="#">08-visual-diagrams.md</a>               | Architecture, user journey, persona mapping, grounded answer, and handoff diagrams            |

### Submission Links

- Live demo: [https://parzival1821.github.io/Multilingual\\_Chatbot/](https://parzival1821.github.io/Multilingual_Chatbot/)
- GitHub repository: [https://github.com/parzival1821/Multilingual\\_Chatbot](https://github.com/parzival1821/Multilingual_Chatbot)
- Problem statement: Track 1 - AI & Intelligent Solutions, Challenge 1.1 - AI-Based Multilingual Chatbot for Welfare Scheme Awareness

### PDF Exports

- Combined PDF packet: [pdf/sahayak-submission-pack.pdf](#)
- Individual PDF exports are available in [pdf/](#).

### Official Deliverable Coverage

| Official deliverable                  | Repo artifact                                                   |
|---------------------------------------|-----------------------------------------------------------------|
| Problem Analysis Report               | <a href="#">01-problem-analysis-report.md</a>                   |
| Solution Framework                    | <a href="#">02-solution-framework.md</a>                        |
| Prototype / Implementation Plan       | <a href="#">03-prototype-implementation-plan.md</a> + live app  |
| Final Presentation                    | <a href="#">06-final-presentation-outline.md</a>                |
| Functional chatbot prototype          | Live demo + source code                                         |
| User-flow diagram covering 3 personas | README Mermaid diagrams + <a href="#">08-visual-diagrams.md</a> |
| Pilot test report                     | <a href="#">05-pilot-test-report.md</a>                         |
| 2-page impact projection              | <a href="#">04-impact-projection.md</a>                         |

### Deadline

Official submission deadline: 13 June 2026, 6:00 PM IST.

# Problem Analysis Report

Project: Sahayak - Welfare Scheme Discovery Assistant

Track: Track 1 - AI & Intelligent Solutions

Challenge: 1.1 - AI-Based Multilingual Chatbot for Welfare Scheme Awareness

Live demo: [https://parzival1821.github.io/Multilingual\\_Chatbot/](https://parzival1821.github.io/Multilingual_Chatbot/)

## Executive Summary

India has a large welfare ecosystem, but eligible beneficiaries often fail to discover, understand, or complete applications for the schemes they qualify for. The official NSS challenge brief identifies awareness gaps, language barriers, low literacy, intimidating documentation, and low-bandwidth access as key bottlenecks. Sahayak addresses this by offering a low-bandwidth, multilingual assistant that asks simple eligibility questions, returns a personalized scheme shortlist, cites official sources, and generates a document checklist that can be copied or downloaded.

The MVP focuses on depth and trust rather than breadth. It covers 8 high-impact schemes and demonstrates the core product loop: profile collection, eligibility matching, grounded follow-up answers, and document handoff.

## Problem Statement

Rural and low-income users face high friction when trying to identify and apply for welfare schemes. Even when schemes exist and funds are allocated, the last-mile user may not know:

- Which schemes apply to their household.
- What eligibility conditions matter.
- Which documents are required.
- Where to apply or verify details.
- Whether the information they receive is reliable.

The challenge is not only a search problem. It is a comprehension, trust, language, and process problem.

## Context From Official Brief

The NSS Challenge 1.1 brief highlights:

- India runs more than 950 central and state welfare schemes.
- Awareness among eligible rural beneficiaries is low.
- Government portals often assume smartphone literacy and stable connectivity.
- Language barriers and documentation complexity reduce successful applications.
- A useful solution should be low-bandwidth, multilingual, grounded in official scheme data, and deployable to channels such as WhatsApp, SMS, or IVR.

## Root Causes

### 1. Information Fragmentation

Scheme details are spread across multiple portals, PDFs, FAQs, state offices, and local intermediaries. Users must know the scheme name before they can search for details.

### 2. Eligibility Confusion

Eligibility is usually described in policy language. A beneficiary thinks in terms of life context: "I am a farmer", "I do not have a gas connection", "I am a widow", or "I need help for my daughter's future." Existing portals often do not translate that context into scheme matches.

### 3. Language and Literacy Barriers

Even when Hindi is available, content may be dense, bureaucratic, or partly in English. Users need simple language, regional-language support, and short explanations.

#### 4. Documentation Friction

Users may know a scheme exists but fail to apply because they do not know what documents to prepare. The checklist is often the point where awareness becomes action.

#### 5. Trust and Hallucination Risk

AI systems can produce confident but incorrect answers. For welfare schemes, wrong eligibility guidance can waste time, money, and trust. A responsible assistant must cite official sources and admit when it does not know.

#### 6. Low-Bandwidth Constraints

Many target users have shared phones, limited data, unstable connectivity, or low-end devices. A text-first flow with copyable/downloadable output is more practical than a heavy app.

### Stakeholders

| Stakeholder                     | Need                                                               |
|---------------------------------|--------------------------------------------------------------------|
| Rural beneficiary               | Simple scheme discovery and document checklist                     |
| Woman-led household             | Awareness of LPG, housing, social security, and girl-child support |
| Street vendor / informal worker | Credit, pension, health, and social protection guidance            |
| NGO field worker                | Fast beneficiary triage during outreach camps                      |
| NSS volunteer                   | A repeatable tool for community scheme-awareness drives            |
| District administration         | Higher uptake without creating new welfare programs                |

### Target Personas

#### Persona 1: Farmer

- Rural household.
- Owns cultivable land.
- Has a bank account.
- Needs income support and health protection.
- Relevant schemes: PM-KISAN, PM-JAY.

#### Persona 2: Woman-led Household

- Urban low-income household.
- Adult woman, no LPG connection.
- May be widowed or guardian of a girl child.
- Relevant schemes: Ujjwala, PMAY-U, Sukanya Samriddhi, NSAP depending on profile.

#### Persona 3: Street Vendor

- Urban or peri-urban informal worker.
- Has a bank account.
- Needs livelihood credit and future pension support.
- Relevant schemes: PM SVANidhi, Atal Pension Yojana, PM-JAY.

### User Needs

Users need a system that can:

1. Ask only a few simple questions.
2. Convert life context into scheme recommendations.
3. Explain why a scheme is relevant.

4. Provide document checklists in the user's language.
5. Cite official sources.
6. Avoid unsupported claims.

### Existing Alternatives and Gaps

| Existing path             | Gap                                                        |
|---------------------------|------------------------------------------------------------|
| Government scheme portals | Accurate but difficult to navigate for first-time users    |
| Search engines            | Require knowing scheme names and reading long pages        |
| Local intermediaries      | Can help, but quality and availability vary                |
| Generic chatbots          | Risk hallucination if not grounded in official scheme data |
| PDF awareness drives      | Static, not personalized, and hard to update               |

### Design Requirements Derived From Analysis

- Focus on 8-10 high-impact schemes for quality over breadth.
- Keep eligibility questions simple.
- Support at least English and Hindi.
- Provide source-backed answers.
- Generate copyable/downloadable document checklists.
- Use a low-bandwidth static web implementation for MVP.
- Add a fallback for unsupported questions.
- Structure the code so additional schemes and languages can be added later.

### Success Criteria

| Criteria         | MVP target                                             |
|------------------|--------------------------------------------------------|
| Scheme coverage  | 8 high-impact schemes                                  |
| Language support | English + Hindi Devanagari                             |
| Completion flow  | User can go from profile to shortlist to checklist     |
| Answer safety    | Unsupported questions return fallback                  |
| Source trust     | Scheme cards and answers include official source links |
| Performance      | Static site loads quickly on low-end connections       |
| Repeatability    | Tests validate recommendation and retrieval behavior   |

### Conclusion

Sahayak reframes welfare awareness as a last-mile product problem. The core insight is that beneficiaries do not need a database dump; they need a short, trusted, language-aware decision flow. A focused, grounded, low-bandwidth assistant can help users move from uncertainty to action by identifying relevant schemes and preparing the documents needed to apply.

# Solution Framework

Project: Sahayak - Welfare Scheme Discovery Assistant  
Challenge: AI-Based Multilingual Chatbot for Welfare Scheme Awareness

## Solution Overview

Sahayak is a static, low-bandwidth web assistant that combines structured eligibility scoring with retrieval-grounded follow-up answers. The system does not attempt to cover every welfare scheme. Instead, it focuses on a curated catalogue of 8 high-impact schemes and demonstrates a trustworthy product loop:

1. Collect simple profile signals.
2. Rank relevant schemes.
3. Explain why each scheme matches.
4. Answer follow-up questions using scheme-grounded retrieval.
5. Generate a document checklist for application preparation.

## Product Flow

Diagram: Rendered in the Visual Diagrams Appendix PDF.

## Technical Architecture

Diagram: Rendered in the Visual Diagrams Appendix PDF.

## Tech Stack

| Layer                 | Implementation                                                                             |
|-----------------------|--------------------------------------------------------------------------------------------|
| Frontend              | HTML, CSS, vanilla JavaScript                                                              |
| Data                  | Curated local JSON-like objects in src/data.js                                             |
| Recommendation engine | Rules-based scoring in src/core.js                                                         |
| Retrieval             | Keyword/token matching over scheme names, tags, summaries, Hindi names, documents, and sea |
| Language support      | English + Hindi Devanagari copy maps                                                       |
| Deployment            | GitHub Pages                                                                               |
| Testing               | Node-based core and static smoke tests                                                     |

## Knowledge Base

The MVP includes 8 schemes:

- PM-KISAN Samman Nidhi.
- Ayushman Bharat PM-JAY.
- Pradhan Mantri Ujjwala Yojana.
- Pradhan Mantri Awas Yojana - Urban 2.0.
- National Social Assistance Programme.
- PM Street Vendor's AtmaNirbhar Nidhi.
- Atal Pension Yojana.
- Sukanya Samriddhi Account.

Each scheme entry stores:

- English name.
- Hindi display name.
- English and Hindi summaries.
- Target beneficiary.
- Benefits.

- Eligibility signals.
- Documents.
- Application paths.
- Official source URLs.
- Tags and search text.
- Verification status.

## Recommendation Logic

The recommendation engine converts a user profile into a ranked shortlist. It evaluates signals such as:

- Rural vs urban area.
- Occupation.
- Gender.
- Age group.
- Income category.
- Cultivable land.
- Permanent house ownership.
- LPG connection.
- Bank account.
- Special categories such as widow, disability, and girl-child guardian.

Each scheme has simple scoring rules. For example:

- PM-KISAN gains score for farmer occupation, cultivable land, and rural area.
- Ujjwala gains score for adult woman, no LPG connection, and low income.
- PMAY-U gains score for urban household, no permanent house, and EWS/LIG/MIG/BPL income category.
- NSAP gains score for BPL status, senior citizen, widowhood, or disability.

## Retrieval-Grounded Answering

The assistant uses retrieval instead of free-form hallucination:

1. Normalize the user's question.
2. Tokenize English and Devanagari text while preserving Hindi matras.
3. Match against scheme name, Hindi name, summary, documents, tags, and search text.
4. Check whether the match has enough context.
5. Compose an answer using only stored scheme data.
6. Attach an official source link.
7. Return a safe fallback if the match is weak.

## Safety and Trust

Sahayak avoids unsupported claims through:

- Official source links on recommendation cards.
- Source links inside assistant answers.
- Verification labels for each scheme entry.
- Explicit recheck labels for schemes where latest rules should be verified.
- Safe fallback for unsupported questions.
- Tests for fallback behavior and high-risk query routing.

## Multilingual Design

The MVP supports:

- English interface.
- Hindi interface in Devanagari.
- Hindi scheme display names.

- Hindi summaries, reasons, document names, benefits, and prompts.
- Devanagari query handling for examples such as:
  - आयुष्मान के लिए कौन से दस्तावेज चाहिए?
  - क्या मैं पीएम-किसान के लिए पात्र हूँ?
  - मैं 55 वर्ष की विधवा हूँ, मेरे लिए कौन सी योजना बेहतर है?

## Low-Bandwidth Design Choices

- Static website with no frontend framework.
- Text-first UI.
- No heavy media dependencies.
- Copyable and downloadable .txt checklists.
- Works as a single-page app hosted on GitHub Pages.
- Future-ready for WhatsApp/SMS/IVR because the logic is text-based.

## Scalability Path

| Phase   | Extension                                                          |
|---------|--------------------------------------------------------------------|
| MVP     | Static web prototype with 8 schemes                                |
| Phase 2 | Add more regional languages and more schemes                       |
| Phase 3 | Replace local retrieval with indexed official scheme data          |
| Phase 4 | Add WhatsApp/SMS interface through Twilio or WhatsApp Business API |
| Phase 5 | Add field-worker dashboard and analytics for outreach camps        |

## Risks and Mitigations

| Risk                   | Mitigation                                      |
|------------------------|-------------------------------------------------|
| Outdated scheme rules  | Verification labels and official source links   |
| Hallucinated answers   | Retrieval-grounded composition and fallback     |
| Low literacy           | Short questions, simple summaries, Hindi labels |
| Poor connectivity      | Static low-bandwidth web app and text checklist |
| Excessive scheme scope | Focused 8-scheme catalogue                      |
| Language gaps          | Copy maps and searchable Hindi display names    |

## Financial and Operational Model

For MVP deployment:

- Hosting cost: free through GitHub Pages.
- Data storage: local static files.
- Maintenance: periodic scheme updates from official portals.
- Field deployment: NSS volunteers or NGO field workers can use it during awareness drives.

For future WhatsApp deployment:

- Twilio/WhatsApp Business API costs depend on message volume.
- A district or NGO deployment can start with a small curated scheme catalogue and expand.
- The assistant can be maintained by a small technical volunteer team.

## Conclusion

Sahayak is feasible because it avoids unnecessary complexity. It demonstrates the core AI product value: grounded personalization, multilingual access, and actionable document preparation. Its architecture is simple enough for a student team to build and maintain, while still extensible toward real last-mile channels.

# Prototype and Implementation Plan

Project: Sahayak - Welfare Scheme Discovery Assistant  
Live demo: [https://parzival1821.github.io/Multilingual\\_Chatbot/](https://parzival1821.github.io/Multilingual_Chatbot/)  
Repository: [https://github.com/parzival1821/Multilingual\\_Chatbot](https://github.com/parzival1821/Multilingual_Chatbot)

## Prototype Status

The current prototype is a deployed static web app. It demonstrates the end-to-end user journey required for the NSS Challenge 1.1 MVP:

1. Select English or Hindi.
2. Fill an eligibility profile.
3. View personalized welfare scheme recommendations.
4. Ask grounded follow-up questions.
5. Generate, copy, and download a document checklist.

## MVP Feature Map

| Feature                      | Status   | Notes                                           |
|------------------------------|----------|-------------------------------------------------|
| Live prototype               | Complete | GitHub Pages deployment                         |
| 8-scheme catalogue           | Complete | Central schemes with official source links      |
| Eligibility questionnaire    | Complete | Uses household and occupation signals           |
| Personalized recommendations | Complete | Ranked cards with reasons                       |
| Hindi interface              | Complete | Devanagari UI, prompts, scheme names, summaries |
| Follow-up assistant          | Complete | Retrieval-grounded answer generation            |
| Safe fallback                | Complete | Prevents unsupported answers                    |
| Document checklist           | Complete | Copy/download text checklist                    |
| Demo personas                | Complete | Farmer, woman-led household, street vendor      |
| Automated tests              | Complete | Core and static smoke checks                    |

## Repository Structure

```
.
├── index.html
├── styles.css
├── package.json
├── README.md
├── src
│   ├── app.js
│   ├── core.js
│   └── data.js
├── tests
│   ├── core.test.js
│   └── static-smoke.test.js
├── submission
│   ├── 01-problem-analysis-report.md
│   ├── 02-solution-framework.md
│   ├── 03-prototype-implementation-plan.md
│   ├── 04-impact-projection.md
│   ├── 05-pilot-test-report.md
│   ├── 06-final-presentation-outline.md
│   ├── 07-submission-portal-details.md
│   └── 08-visual-diagrams.md
```

## Key Files

| File                       | Responsibility                                                     |
|----------------------------|--------------------------------------------------------------------|
| index.html                 | Dashboard structure                                                |
| styles.css                 | Responsive visual design                                           |
| src/data.js                | Scheme knowledge base and language labels                          |
| src/core.js                | Recommendation, retrieval, profile inference, checklist formatting |
| src/app.js                 | UI event handling and rendering                                    |
| tests/core.test.js         | Behavioral tests for recommendations, assistant, checklist         |
| tests/static-smoke.test.js | Static app and deployment checks                                   |

## Implementation Flow

Diagram: Rendered in the Visual Diagrams Appendix PDF.

## Local Run Instructions

`npm run dev`

Then open:

`http://localhost:4173`

## Test Instructions

Run core tests:

`npm test`

Run all checks:

`npm run check`

Expected result:

All core tests passed.

Static smoke checks passed.

## Deployment

The project is deployed using GitHub Pages from the main branch.

Deployment workflow:

1. Push changes to main.
2. GitHub Actions builds the static site.
3. Pages serves the app at the live demo URL.

## Demo Script

1. Open the live app.
2. Use English mode first.
3. Click the Farmer preset.
4. Show PM-KISAN as a strong match.
5. Click View documents.
6. Copy/download the checklist.
7. Ask: What documents do I need for Ayushman Bharat?
8. Switch to Hindi.
9. Ask: मुझे घर के लिए कौन सी योजना मिल सकती है?
10. Show Hindi recommendations, source links, and fallback behavior.

## Prototype Validation

Automated checks currently cover:

- PM-KISAN ranking for farmer profile.
- PM-JAY retrieval for Ayushman document questions.
- Hindi Devanagari retrieval.
- Widow follow-up routing to NSAP.
- PM SVANidhi ranking for street vendor profile.
- Checklist generation and formatting.
- Safe fallback for unsupported queries.
- Static asset references.
- GitHub Pages workflow shape.

## Implementation Roadmap

Immediate Submission Version

- Keep the current 8-scheme web MVP.
- Use README and visual appendix diagrams for architecture and user flow.
- Submit GitHub repo and live demo link.
- Use the internal validation report, then add real-user observations if time permits.

Next Version

- Add one more regional language beyond Hindi.
- Add WhatsApp/SMS interface through Twilio or WhatsApp Business API.
- Add admin tool for updating scheme data.
- Expand scheme coverage using official scheme catalogues.
- Add analytics for NGO/NSS outreach camps.

## Conclusion

The current prototype is ready as a functional MVP. It demonstrates the core end-to-end solution loop and is simple enough to deploy, test, and explain during evaluation.

# Impact Projection

Project: Sahayak - Welfare Scheme Discovery Assistant

## Objective

This document estimates the potential social impact of Sahayak if used by NSS volunteers, NGOs, or local field workers during welfare awareness drives. The projection is conservative and intended to show the order of magnitude of possible benefit, not claim measured deployment impact.

## Impact Logic

Sahayak improves outcomes by reducing friction in four steps:

1. Awareness: user discovers relevant schemes.
2. Comprehension: user understands why the scheme may apply.
3. Preparation: user receives a document checklist.
4. Action: user is more likely to apply through official channels.

Diagram: Rendered in the Visual Diagrams Appendix PDF.

## Base Assumptions

| Variable                                                | Conservative assumption                          |
|---------------------------------------------------------|--------------------------------------------------|
| Outreach population in pilot district/NGO program       | 10,000 people                                    |
| Share likely eligible for at least one listed scheme    | 40%                                              |
| Eligible people in target population                    | 4,000                                            |
| Current awareness/completion gap                        | Significant, based on official challenge framing |
| Additional users who attempt application due to Sahayak | 10-20% of eligible users                         |
| Successful completion among additional attempts         | 40-60%                                           |

## Scenario Model

### Conservative Scenario

- Eligible users: 4,000.
- Additional application attempts due to Sahayak: 10% = 400.
- Successful completion rate among additional attempts: 40%.
- Additional successful enrollments/applications: 160.

### Moderate Scenario

- Eligible users: 4,000.
- Additional application attempts due to Sahayak: 15% = 600.
- Successful completion rate among additional attempts: 50%.
- Additional successful enrollments/applications: 300.

### High Scenario

- Eligible users: 4,000.
- Additional application attempts due to Sahayak: 20% = 800.
- Successful completion rate among additional attempts: 60%.
- Additional successful enrollments/applications: 480.

## Entitlement Value Illustration

The value differs by scheme. The table below uses illustrative benefits from the current catalogue and should be verified scheme-by-scheme before field reporting.

| Scheme      | Benefit type               | Example value mechanism                                |
|-------------|----------------------------|--------------------------------------------------------|
| PM-KISAN    | Direct income support      | Rs. 6,000 per year for eligible farmer families        |
| PM-JAY      | Health protection          | Up to Rs. 5 lakh annual family health cover            |
| Ujjwala     | Clean cooking fuel support | Deposit-free LPG connection and related support        |
| PMAY-U      | Housing support            | Housing construction/purchase/rental assistance        |
| NSAP        | Social assistance          | Pension or family benefit support                      |
| PM SVANidhi | Livelihood credit          | Working capital loan and incentives                    |
| APY         | Pension                    | Guaranteed pension after age 60 based on contributions |
| Sukanya     | Savings                    | Government-backed girl-child savings account           |

## PM-KISAN Example Calculation

If only 50 of the additional successful applications are PM-KISAN enrollments:

50 households x Rs. 6,000 per year = Rs. 3,00,000 per year

If 200 households are helped over a larger outreach cycle:

200 households x Rs. 6,000 per year = Rs. 12,00,000 per year

This calculation covers only one scheme. Health coverage, housing support, pension support, and livelihood credit can create additional financial protection beyond direct cash transfer.

## Time and Process Savings

Sahayak can also reduce non-cash friction:

| Friction point              | Expected improvement                           |
|-----------------------------|------------------------------------------------|
| Finding relevant schemes    | One guided profile instead of manual searching |
| Understanding eligibility   | Short reason bullets per recommendation        |
| Preparing documents         | Copyable/downloadable checklist                |
| Avoiding wrong applications | Better profile-to-scheme matching              |
| Field-worker triage         | Faster beneficiary screening during camps      |

If a field worker saves even 10 minutes per beneficiary screened, then:

1,000 screened users x 10 minutes = 10,000 minutes saved

10,000 minutes = 166.7 hours saved

## Cost Projection

MVP Cost

| Cost head          | Estimate                 |
|--------------------|--------------------------|
| Hosting            | Free on GitHub Pages     |
| Frontend framework | None                     |
| Data storage       | Static files             |
| Maintenance        | Volunteer/developer time |
| Field usage        | Any phone/browser        |

## Future WhatsApp/SMS Cost

Costs would depend on message volume and provider pricing. The current architecture keeps the core logic text-based, so the same recommendation and retrieval engine can later be exposed through a WhatsApp/SMS backend.

## Sustainability Pathway

1. NSS volunteers use the tool during awareness camps.
2. Local NGOs validate scheme coverage and language clarity.
3. Additional schemes/languages are added based on local need.
4. District-level or NGO-level deployment integrates WhatsApp/SMS.
5. Field feedback updates scheme data and document requirements.

## Impact Metrics to Track in Pilot

| Metric                      | Why it matters                                     |
|-----------------------------|----------------------------------------------------|
| Flow completion rate        | Measures usability                                 |
| Comprehension score         | Measures whether users understood scheme summaries |
| Checklist usefulness rating | Measures actionability                             |
| Application intent          | Measures likely next step                          |
| Actual application attempt  | Measures conversion                                |
| Successful enrollment       | Measures final welfare uptake                      |

## Conclusion

Sahayak's impact comes from increasing successful navigation of existing welfare schemes. Even modest improvements in awareness and document readiness can translate into meaningful entitlement access, especially when the tool is used by NSS volunteers or NGOs across repeated community outreach sessions.

# Pilot Test Report and Internal Validation

Project: Sahayak - Welfare Scheme Discovery Assistant  
Official expected deliverable: Pilot test report with 10-15 real users from any nearby village or community

## Evidence Status

This report is an honest internal validation of the current prototype. It uses scripted beneficiary personas, automated tests, and manual walkthroughs completed by the project team on 13 June 2026.

It does not claim feedback from 10-15 external community users. If a field pilot or short demo is completed before final submission, the real-user appendix at the end of this document can be filled in directly.

## Validation Objective

- Evaluate whether Sahayak can help a user:
1. Enter an eligibility profile in English or Hindi.
  2. Receive relevant welfare scheme recommendations.
  3. Understand why a scheme was suggested.
  4. Ask a follow-up question.
  5. Prepare a document checklist for application.
  6. Verify the result through official source links.

## Test Setup

| Field             | Details                                                                                                                 |
|-------------------|-------------------------------------------------------------------------------------------------------------------------|
| Date              | 13 June 2026                                                                                                            |
| Tester type       | Internal project validation                                                                                             |
| Prototype         | Static GitHub Pages web app                                                                                             |
| Live URL          | <a href="https://parzival1821.github.io/Multilingual_Chatbot/">https://parzival1821.github.io/Multilingual_Chatbot/</a> |
| Repository        | <a href="https://github.com/parzival1821/Multilingual_Chatbot">https://github.com/parzival1821/Multilingual_Chatbot</a> |
| Languages checked | English, Hindi in Devanagari                                                                                            |
| Test method       | Scripted persona walkthroughs + automated Node.js checks                                                                |
| Evidence files    | tests/core.test.js, tests/static-smoke.test.js, README, submission docs                                                 |

## Scripted Persona Coverage

| Persona                | Key profile signals                                                           | Expected scheme    |
|------------------------|-------------------------------------------------------------------------------|--------------------|
| Landholding farmer     | Rural, farmer, cultivable land, bank account                                  | PM-KISAN, PM-J     |
| Woman-led household    | Female applicant, low income, no LPG, no permanent house, girl child guardian | Ujjwala, PMAY-U    |
| Street vendor          | Urban street vendor, low income, bank account                                 | PM SVANidhi, PM    |
| Widow aged 55          | Widow, BPL/low income, social assistance need                                 | NSAP               |
| Health-document seeker | Asks for Ayushman Bharat documents                                            | PM-JAY checklis    |
| Unsupported query      | Asks outside current knowledge base                                           | Safe fallback with |

## Manual Walkthrough Results

| Test task               | Expected behavior                                          |
|-------------------------|------------------------------------------------------------|
| Open the live prototype | App loads without login and shows Sahayak dashboard        |
| Switch to Hindi         | UI copy, prompt examples, and scheme names use Devanagari  |
| Use Farmer demo preset  | Form fills a farmer profile without hiding editable fields |

| Test task                                             | Expected behavior                                                       |
|-------------------------------------------------------|-------------------------------------------------------------------------|
| Click Find Schemes                                    | Recommendations are ranked with reasons and source links                |
| Click View documents                                  | Checklist panel updates to the selected scheme and scrolls into view    |
| Ask What documents do I need for Ayushman Bharat?     | Assistant returns PM-JAY document guidance with source link             |
| Ask I am widow of age 55 which scheme is best for me? | Assistant routes to NSAP social assistance guidance                     |
| Copy checklist                                        | Checklist text copies in a low-bandwidth-friendly format                |
| Download checklist                                    | Browser downloads a plain-text checklist                                |
| Ask unsupported startup subsidy query                 | Assistant avoids hallucination and asks user to verify official sources |

## Automated Validation

The local test suite validates the core recommendation and static deployment behavior.

| Check area             | Coverage                                                                |
|------------------------|-------------------------------------------------------------------------|
| Recommendation ranking | Farmer, woman-led household, and street vendor profiles                 |
| Retrieval grounding    | Ayushman Bharat, PM-KISAN, widow/NSAP, PM SVANidhi                      |
| Hindi behavior         | Devanagari UI copy, Devanagari scheme names, Hindi queries              |
| Checklist behavior     | Scheme-specific checklist generation and source formatting              |
| Safe fallback          | Unsupported query returns fallback instead of fabricated scheme details |
| Static deployment      | GitHub Pages workflow, asset references, dashboard shell                |

Expected local verification command:

```
npm run check
```

Expected result:

All core tests passed.

Static smoke checks passed.

## Internal Metrics

These are prototype-validation metrics, not external community-pilot metrics.

| Metric                           | Internal result           | Notes                                                       |
|----------------------------------|---------------------------|-------------------------------------------------------------|
| Scripted flow completion         | 6/6 scenarios passed      | All scripted personas reached a useful result               |
| Recommendation correctness proxy | 6/6 scenarios passed      | Based on expected scheme-persona mapping                    |
| Checklist availability           | 8/8 schemes covered       | Every scheme has application documents                      |
| Source grounding                 | 8/8 schemes covered       | Every scheme has at least one official source link          |
| Language coverage                | 2 languages               | English and Hindi Devanagari                                |
| Response speed                   | Not formally instrumented | Static local/browser interactions appeared immediate during |

## Findings

1. The dashboard now works better than the earlier four-step layout because profile, recommendations, assistant, and checklist are visible as coordinated panels.
2. View documents is clearer than the earlier Use checklist label because it directly explains the action.
3. Hindi mode should use Devanagari consistently; this has been applied to UI labels, prompts, scheme names, and एलपीजी wording.
4. Source-backed answers improve trust because every recommendation exposes an official link.
5. The largest remaining validation gap is external field feedback from real beneficiaries or NSS/community volunteers.

## Improvements Made After Validation

| Issue noticed                                       | Change made                                                       |
|-----------------------------------------------------|-------------------------------------------------------------------|
| Four-step page felt rough and beginner-like         | Reorganized into a single dashboard with dedicated panels         |
| Checklist action did not communicate behavior       | Changed to View documents and connected it to checklist selection |
| Hindi mode used Hinglish in places                  | Replaced with Devanagari copy                                     |
| Scheme names stayed in English in Hindi mode        | Added Hindi display names for schemes                             |
| Local terminology like pucca was unclear in English | Reworded to permanent house                                       |

## Real-User Pilot Appendix

Use this table only if a real demo or field test is completed. Do not fill it with assumed data.

| User ID | Persona / background | Language     | Completed flow? | Top recommendation understood? | Checklist useful? | N |
|---------|----------------------|--------------|-----------------|--------------------------------|-------------------|---|
| U01     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U02     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U03     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U04     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U05     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U06     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U07     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U08     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U09     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U10     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U11     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U12     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U13     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U14     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |
| U15     | To be filled         | To be filled | Yes/No          | Yes/No                         | 1-5               | N |

## Conclusion

The current prototype passes internal validation for the submission demo: it can recommend schemes, explain match reasons, answer grounded follow-up questions, and produce document checklists in English and Hindi.

For a stricter interpretation of the official pilot requirement, the remaining step is to run the same script with 10-15 real users or community representatives and paste observations into the appendix above.

# Final Presentation Outline

Project: Sahayak - Welfare Scheme Discovery Assistant

Recommended length: 6-8 slides

## Slide 1: Title

Sahayak: Multilingual Welfare Scheme Discovery Assistant

Subtitle:

Low-bandwidth, source-backed scheme awareness for rural and low-income beneficiaries.

Include:

- Team/member names.
- NSS Open Projects 2026.
- Track 1.1: AI-Based Multilingual Chatbot for Welfare Scheme Awareness.
- Live demo link and QR code if possible.

Speaker note:

Sahayak helps users discover welfare schemes, understand eligibility, and prepare documents in English or Hindi.

## Slide 2: Problem

Main message:

India has many welfare schemes, but eligible users often do not discover or complete applications.

Use 3 pain points:

- Information is fragmented across portals and PDFs.
- Eligibility language is hard to understand.
- Documentation friction stops users from applying.

Speaker note:

The problem is not lack of welfare design; it is last-mile access, language, trust, and actionability.

## Slide 3: Target Users

Show 3 personas:

1. Farmer.
2. Woman-led household.
3. Street vendor / informal worker.

For each:

- Need.
- Relevant schemes.
- Product value.

Speaker note:

The product is designed around common beneficiary contexts rather than scheme names.

## Slide 4: Solution

Show the user journey:

Diagram: Rendered in the Visual Diagrams Appendix PDF.

Key points:

- 4-6 simple eligibility signals.
- Ranked recommendations.
- Source-backed follow-up answers.
- Copy/download checklist.

Speaker note:

Sahayak turns eligibility confusion into a short decision flow.

## Slide 5: Product Demo

Demo flow:

1. Choose Farmer preset.
2. Show PM-KISAN recommendation.
3. Click View documents.
4. Ask Ayushman document question.
5. Switch to Hindi and ask a housing question.
6. Show fallback for unsupported question.

Speaker note:

Highlight trust markers: official source links, verification labels, and safe fallback.

## Slide 6: Technical Architecture

Show:

- Static dashboard.
- Scheme knowledge base.
- Recommendation engine.
- Retrieval layer.
- Answer composer.
- Checklist formatter.

Speaker note:

The MVP uses retrieval-grounded generation rather than an unconstrained chatbot, reducing hallucination risk.

## Slide 7: Impact Projection

Use conservative model:

- 10,000 outreach population.
- 4,000 likely eligible for at least one scheme.
- 10-20% additional application attempts.
- 160-480 additional successful applications depending on scenario.

Example:

50 additional PM-KISAN enrollments x Rs. 6,000/year = Rs. 3,00,000/year

Speaker note:

Even small improvements in welfare uptake can translate into meaningful entitlement access.

## Slide 8: Roadmap and Ask

Roadmap:

- Add more regional languages.
- Add WhatsApp/SMS/IVR interface.
- Expand scheme catalogue.

- Pilot with NSS/NGO field workers.
- Build update workflow for official scheme changes.

Ask:

- Mentor feedback.
- Pilot access through NSS/NGO community.
- Support for WhatsApp/SMS deployment.

Speaker note:

The prototype is ready for validation and extension into real last-mile channels.

## Backup Slide: Evaluation Fit

| Rubric area           | Sahayak response                                           |
|-----------------------|------------------------------------------------------------|
| Problem understanding | Targets awareness, language, documentation, and trust gaps |
| Innovation            | Grounded welfare assistant with checklist handoff          |
| Feasibility           | Static low-cost MVP deployed on GitHub Pages               |
| Impact potential      | Quantified entitlement and time-saving model               |
| Communication         | Live demo, diagrams, and source-backed documentation       |

## Submission Portal Details

Use this file to fill the NSS submission form.

### Participant Details

| Field               | Value        |
|---------------------|--------------|
| Name of Participant | To be filled |
| Department & Year   | To be filled |
| Team name           | To be filled |
| Team members        | To be filled |

### Problem Statement Chosen

Track 1 - AI & Intelligent Solutions

Challenge 1.1 - AI-Based Multilingual Chatbot for Welfare Scheme Awareness

### Submission Link

Recommended submission link:

[https://github.com/parzival1821/Multilingual\\_Chatbot](https://github.com/parzival1821/Multilingual_Chatbot)

Live prototype:

[https://parzival1821.github.io/Multilingual\\_Chatbot/](https://parzival1821.github.io/Multilingual_Chatbot/)

### Public Access Checklist

- GitHub repository is public.
- GitHub Pages link opens without login.
- README contains live demo link.
- submission/ folder contains supporting deliverables.
- Local-only planning files are ignored and not submitted.

### Deliverable Checklist

| Deliverable                             | Included? | Location                                       |
|-----------------------------------------|-----------|------------------------------------------------|
| Problem Analysis Report                 | Yes       | submission/01-problem-analysis-report.md       |
| Solution Framework                      | Yes       | submission/02-solution-framework.md            |
| Prototype / Implementation Plan         | Yes       | submission/03-prototype-implementation-plan.md |
| Functional prototype                    | Yes       | Live GitHub Pages link                         |
| User-flow diagram                       | Yes       | README and submission/08-visual-diagrams.md    |
| Pilot test report / internal validation | Yes       | submission/05-pilot-test-report.md             |
| 2-page impact projection                | Yes       | submission/04-impact-projection.md             |
| Final presentation outline              | Yes       | submission/06-final-presentation-outline.md    |
| Visual diagrams appendix                | Yes       | submission/08-visual-diagrams.md               |

### Deadline

Official deadline: 13 June 2026, 6:00 PM IST.

## Final Pre-Submit Checklist

- Open live demo and confirm it loads.
- Run npm run check.
- Verify README and visual appendix diagrams render on GitHub.
- Confirm submission folder appears on GitHub.
- Confirm no private/personal notes are committed.
- Submit the GitHub repository URL in the portal.
- If submitting as a team, every team member should submit individually with the same link.

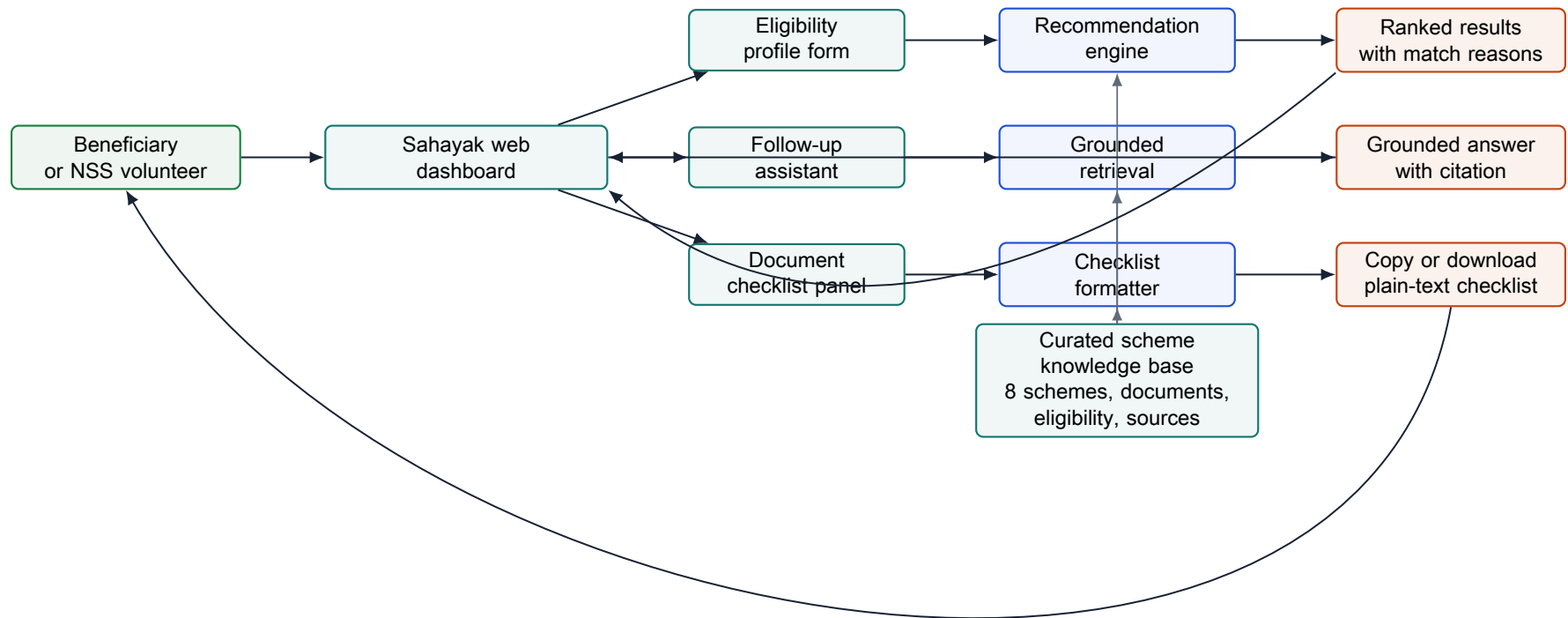
# Sahayak Visual Diagrams Appendix

## Welfare Scheme Discovery Assistant

Architecture, user journey, persona mapping, grounded answer flow, low-bandwidth handoff, and submission map.

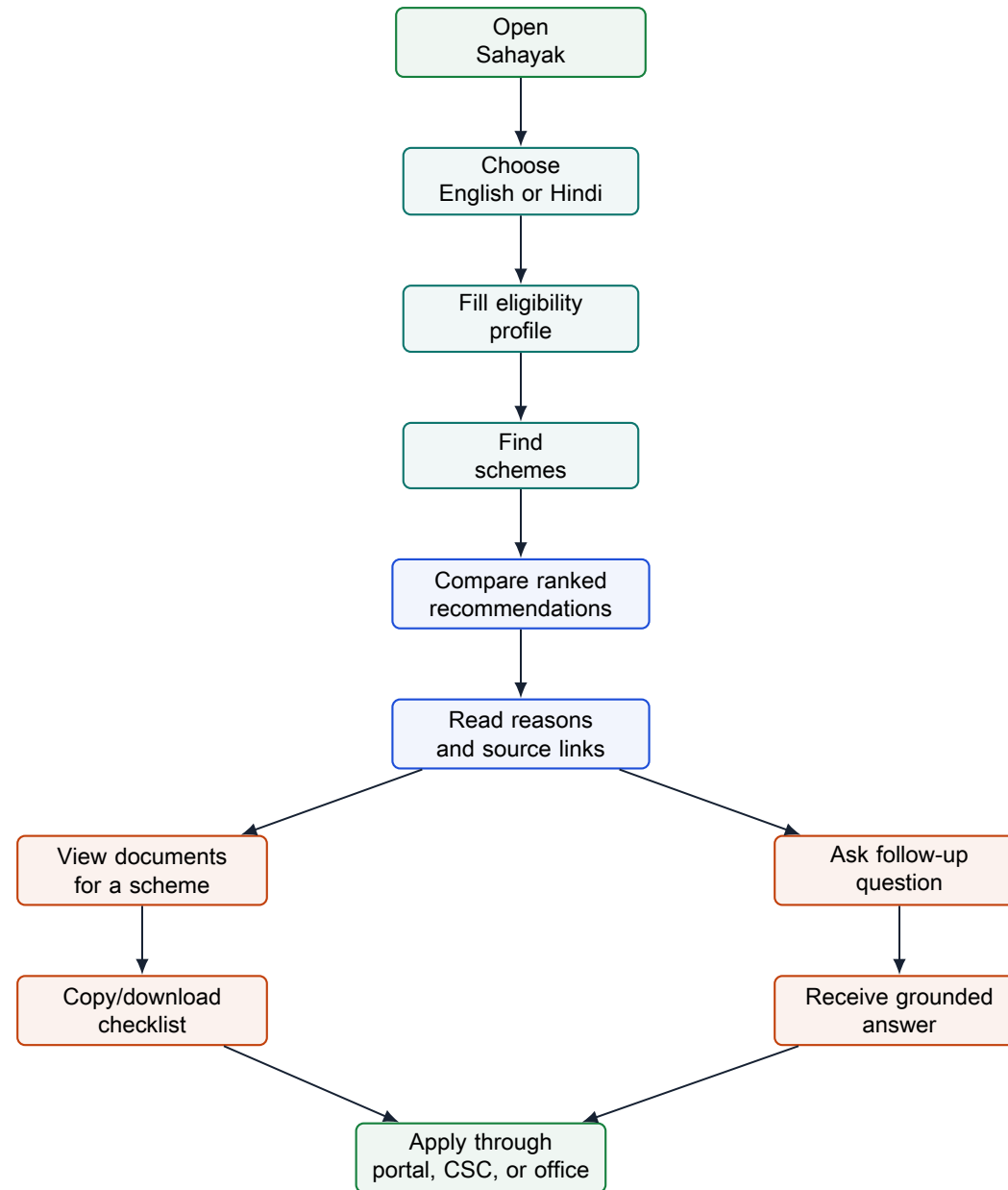
### 1. System Architecture

How the static dashboard, recommendation logic, source-backed retrieval, and checklist actions work together.



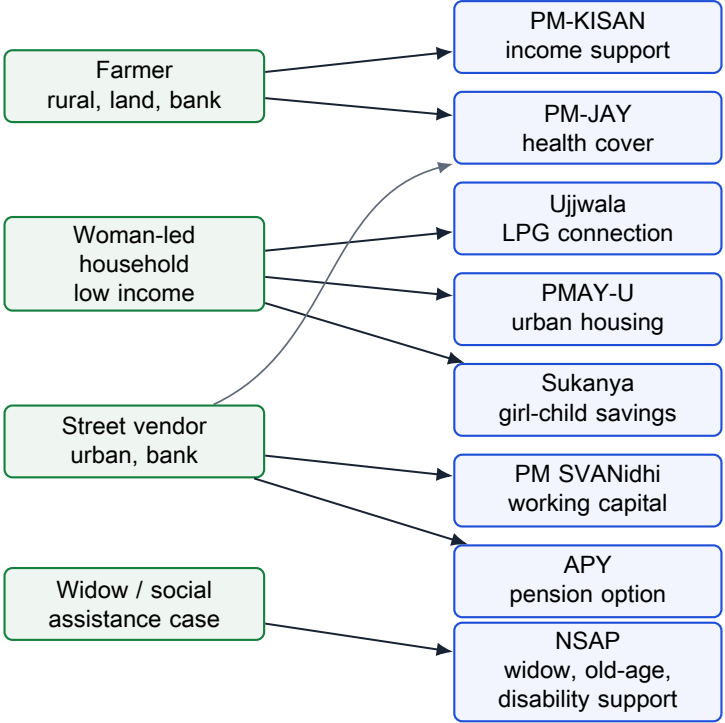
## 2. Beneficiary Journey

The user moves from basic profile entry to scheme discovery, verification, and application preparation.



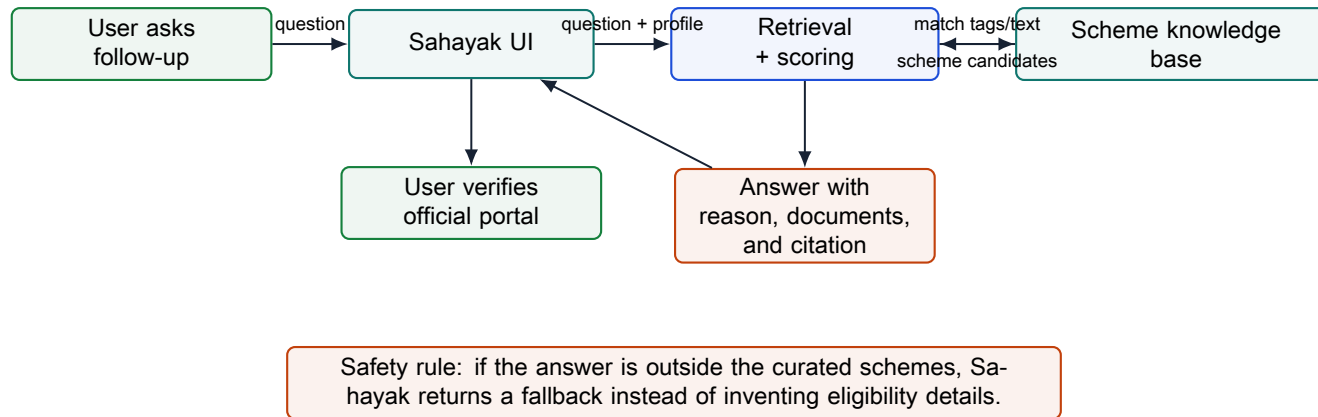
### 3. Persona-To-Scheme Coverage

Three required personas plus social-assistance and health-document scenarios are mapped to the current 8-scheme knowledge base.



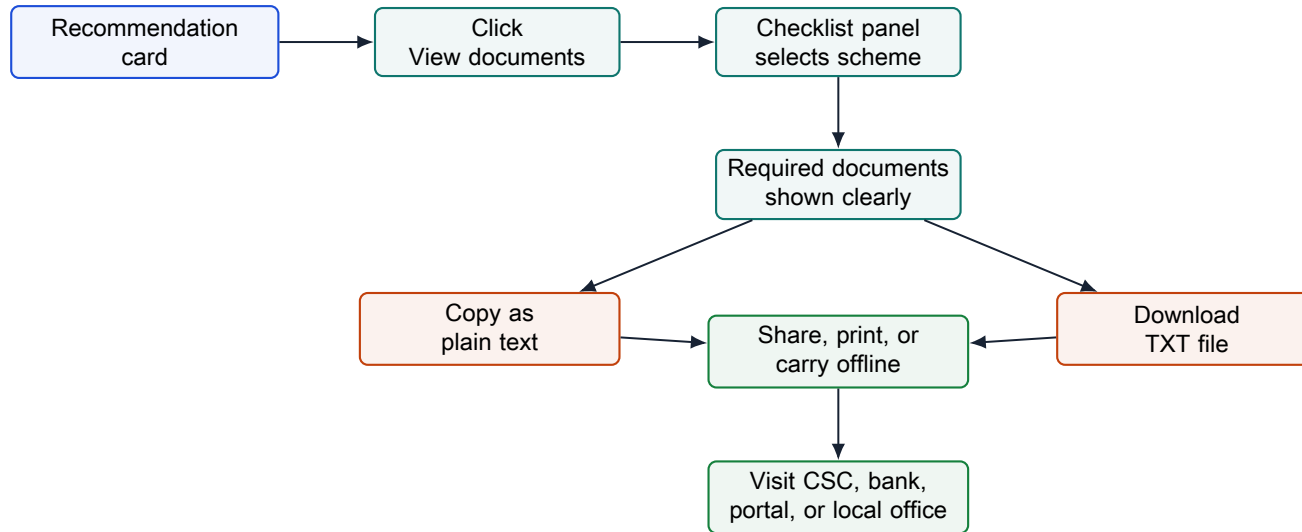
## 4. Grounded Answer Flow

Follow-up answers are limited to the curated knowledge base and always surface an official source for verification.



## 5. Low-Bandwidth Handoff Flow

The checklist output is intentionally plain text so it can move across phones, WhatsApp, SMS, printouts, or field notes.



## 6. Submission Deliverable Map

How the repository artifacts line up with the official NSS submission packet.

